



```
In [ ]: #without using KMeans library
import numpy as np
import matplotlib.pyplot as plt

# Data points
points = np.array([[2, 10], [2, 5], [8, 4], [5, 8], [7, 5], [6, 4], [1, 2], [4, 10]])
initial_centers = np.array([[2, 10], [5, 8], [1, 2]])

# Function to calculate the distance between points
def euclidean_distance(a, b):
    return np.linalg.norm(a - b)

# Function to assign points to the nearest cluster center
def assign_clusters(points, centers):
    clusters = []
    for point in points:
        distances = [euclidean_distance(point, center) for center in centers]
        cluster = np.argmin(distances)
        clusters.append(cluster)
    return clusters

# Function to update the cluster centers
def update_centers(points, clusters, k):
    new_centers = []
    for i in range(k):
        cluster_points = points[np.array(clusters) == i]
        if len(cluster_points) > 0:
            new_center = cluster_points.mean(axis=0)
        else:
            new_center = initial_centers[i]
        new_centers.append(new_center)
    return np.array(new_centers)

# Function to check if centers have converged
def has_converged(old_centers, new_centers):
    return np.array_equal(old_centers, new_centers)

# Number of clusters
k = 3

# K-means clustering algorithm
def kmeans(points, initial_centers, k):
    centers = initial_centers
    clusters = assign_clusters(points, centers)
    converged = False
    while not converged:
        new_centers = update_centers(points, clusters, k)
        converged = has_converged(centers, new_centers)
        centers = new_centers
        clusters = assign_clusters(points, centers)
    return centers, clusters

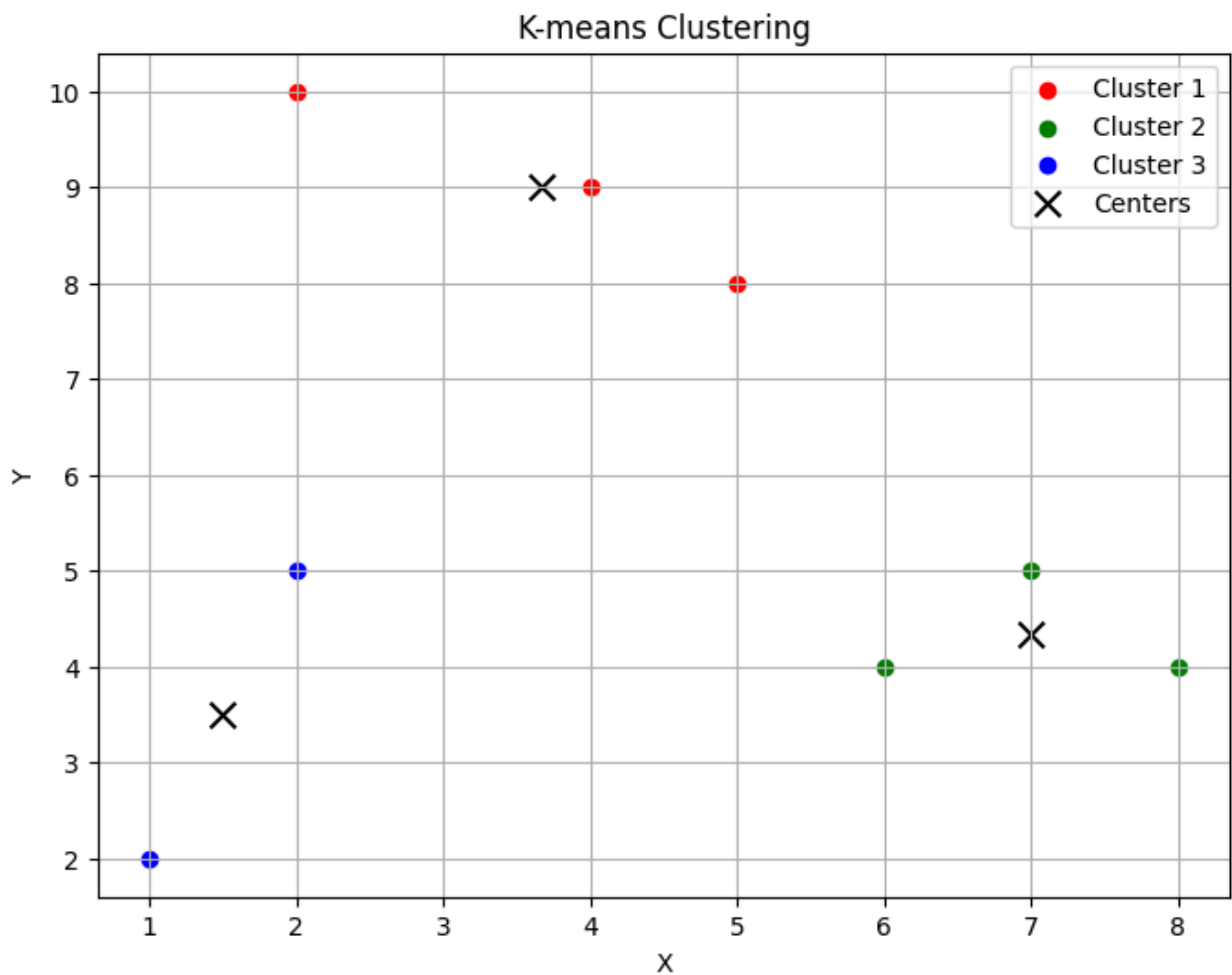
# Running K-means clustering
```

```

final_centers, final_clusters = kmeans(points, initial_centers, k)

# Plotting the results
colors = ['r', 'g', 'b']
plt.figure(figsize=(8, 6))
for i in range(k):
    cluster_points = points[np.array(final_clusters) == i]
    plt.scatter(cluster_points[:, 0], cluster_points[:, 1], c=colors[i], label=i)
plt.scatter(final_centers[:, 0], final_centers[:, 1], c='black', marker='x', s=100)
plt.title('K-means Clustering')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid(True)
plt.show()

```



```

In [ ]: #using KMeans library
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Data points
points = np.array([[2, 10], [2, 5], [8, 4], [5, 8], [7, 5], [6, 4], [1, 2], [4

```

```

# Initial cluster centers
initial_centers = np.array([[2, 10], [5, 8], [1, 2]])

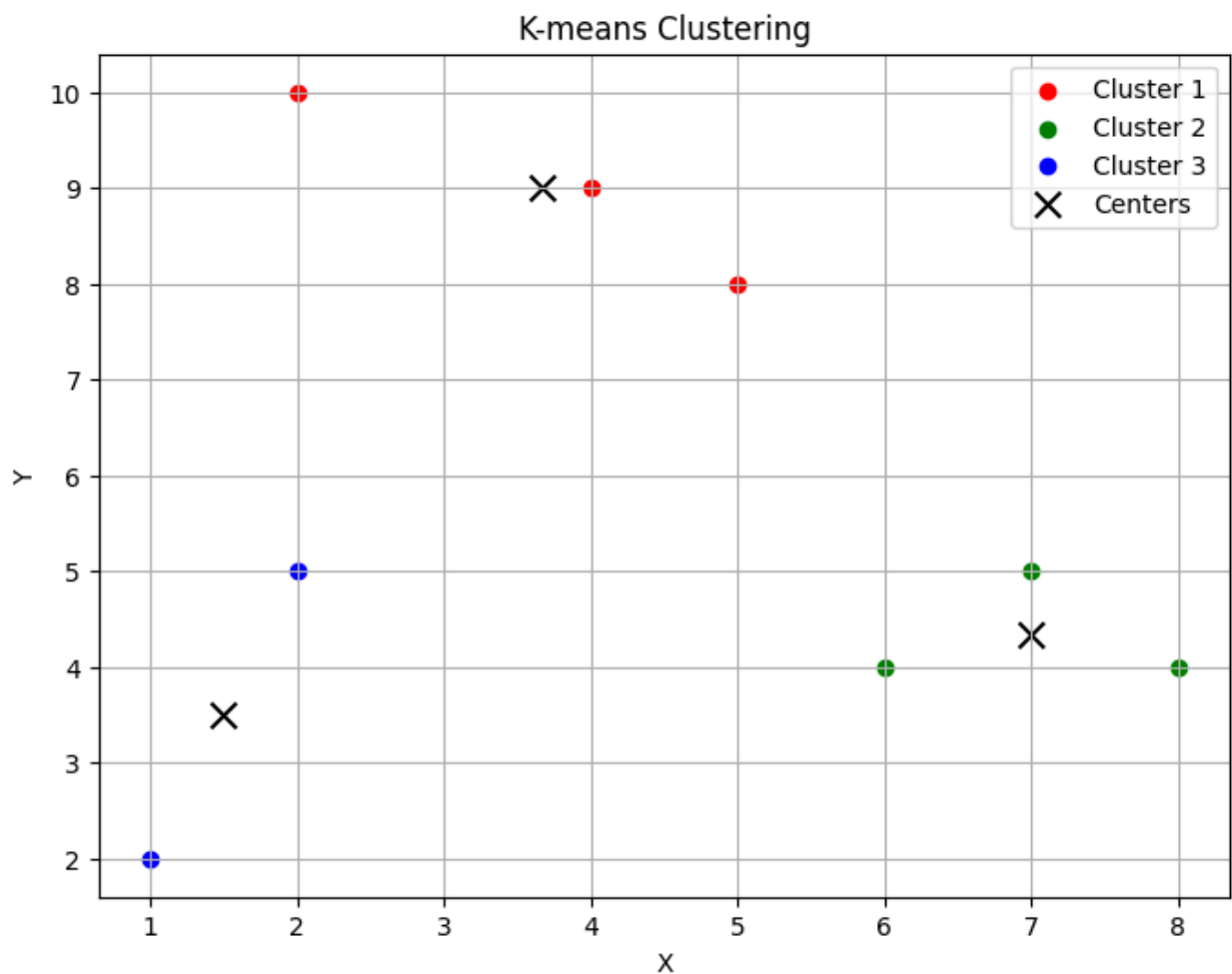
# Number of clusters
k = 3

# KMeans clustering using sklearn
kmeans = KMeans(n_clusters=k, init=initial_centers, n_init=1)
kmeans.fit(points)

# Cluster centers and labels
final_centers = kmeans.cluster_centers_
final_clusters = kmeans.labels_

# Plotting the results
colors = ['r', 'g', 'b']
plt.figure(figsize=(8, 6))
for i in range(k):
    cluster_points = points[final_clusters == i]
    plt.scatter(cluster_points[:, 0], cluster_points[:, 1], c=colors[i], label=i)
plt.scatter(final_centers[:, 0], final_centers[:, 1], c='black', marker='x', s=100)
plt.title('K-means Clustering')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid(True)
plt.show()

```



```
In [ ]: #KMeans clustering using make_blobs dataset
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generate synthetic dataset
n_samples = 300
n_features = 2
n_clusters = 3
random_state = 42

points, true_labels = make_blobs(n_samples=n_samples, n_features=n_features, c

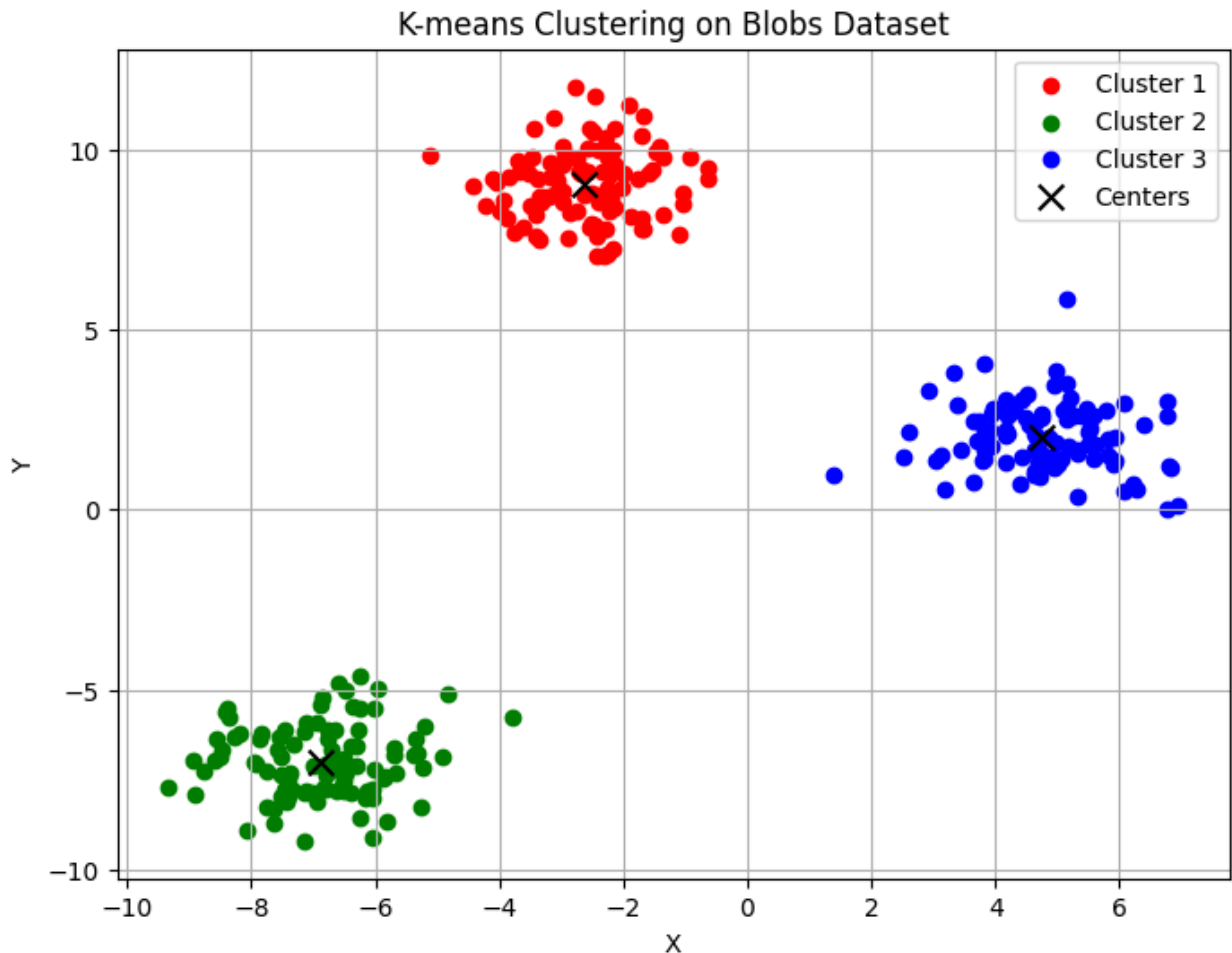
# KMeans clustering using sklearn
kmeans = KMeans(n_clusters=n_clusters, random_state=random_state)
kmeans.fit(points)

# Cluster centers and labels
final_centers = kmeans.cluster_centers_
final_clusters = kmeans.labels_

# Plotting the results
```

```
plt.figure(figsize=(8, 6))
colors = ['r', 'g', 'b']
for i in range(n_clusters):
    cluster_points = points[final_clusters == i]
    plt.scatter(cluster_points[:, 0], cluster_points[:, 1], c=colors[i], label=i)
plt.scatter(final_centers[:, 0], final_centers[:, 1], c='black', marker='x', s=100)
plt.title('K-means Clustering on Blobs Dataset')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid(True)
plt.show()
```

/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:1416: FutureWarning: The default value of `n\_init` will change from 10 to 'auto' in 1.4. Set the value of `n\_init` explicitly to suppress the warning
 super()._check_params_vs_input(X, default_n_init=10)



```
In [ ]: import numpy as np
from sklearn.cluster import KMeans

# Define the points
points = np.array([[2, 10], [2, 5], [8, 4], [5, 8],
                   [7, 5], [6, 4], [1, 2], [4, 9]])
```

```

# Define initial cluster centers
initial_centers = np.array([[2, 10], [5, 8], [1, 2]])

# Apply KMeans with custom initialization
kmeans = KMeans(n_clusters=3, init=initial_centers, n_init=1, random_state=42)
kmeans.fit(points)

# Get results
labels = kmeans.labels_
centers = kmeans.cluster_centers_

# Print results
print("Points:")
point_names = ['A1', 'A2', 'A3', 'A4', 'A5', 'A6', 'A7', 'A8']
for i, (name, point, label) in enumerate(zip(point_names, points, labels)):
    print(f"{name}{tuple(point)} -> Cluster {label}")

print("\nCluster assignments:")
for cluster_id in range(3):
    cluster_points = [point_names[i] for i in range(len(points)) if labels[i] == cluster_id]
    print(f"Cluster {cluster_id}: {cluster_points}")

print("\nFinal cluster centers:")
for i, center in enumerate(centers):
    print(f"Cluster {i}: ({center[0]:.2f}, {center[1]:.2f})")

print(f"\nNumber of iterations: {kmeans.n_iter_}")

```

```

Points:
A1(np.int64(2), np.int64(10)) -> Cluster 0
A2(np.int64(2), np.int64(5)) -> Cluster 2
A3(np.int64(8), np.int64(4)) -> Cluster 1
A4(np.int64(5), np.int64(8)) -> Cluster 0
A5(np.int64(7), np.int64(5)) -> Cluster 1
A6(np.int64(6), np.int64(4)) -> Cluster 1
A7(np.int64(1), np.int64(2)) -> Cluster 2
A8(np.int64(4), np.int64(9)) -> Cluster 0

```

```

Cluster assignments:
Cluster 0: ['A1', 'A4', 'A8']
Cluster 1: ['A3', 'A5', 'A6']
Cluster 2: ['A2', 'A7']

```

```

Final cluster centers:
Cluster 0: (3.67, 9.00)
Cluster 1: (7.00, 4.33)
Cluster 2: (1.50, 3.50)

```

```

Number of iterations: 4

```

```

In [ ]: import numpy as np
        from sklearn.cluster import KMeans
        import matplotlib.pyplot as plt

```

```

# Data
points = np.array([[2,10], [2,5], [8,4], [5,8], [7,5], [6,4], [1,2], [4,9]])
point_names = ['A1', 'A2', 'A3', 'A4', 'A5', 'A6', 'A7', 'A8']
initial_centers = np.array([[2,10], [5,8], [1,2]])

# KMeans clustering
kmeans = KMeans(n_clusters=3, init=initial_centers, n_init=1)
kmeans.fit(points)

# Visualization
plt.figure(figsize=(10, 6))
colors = ['red', 'blue', 'green']

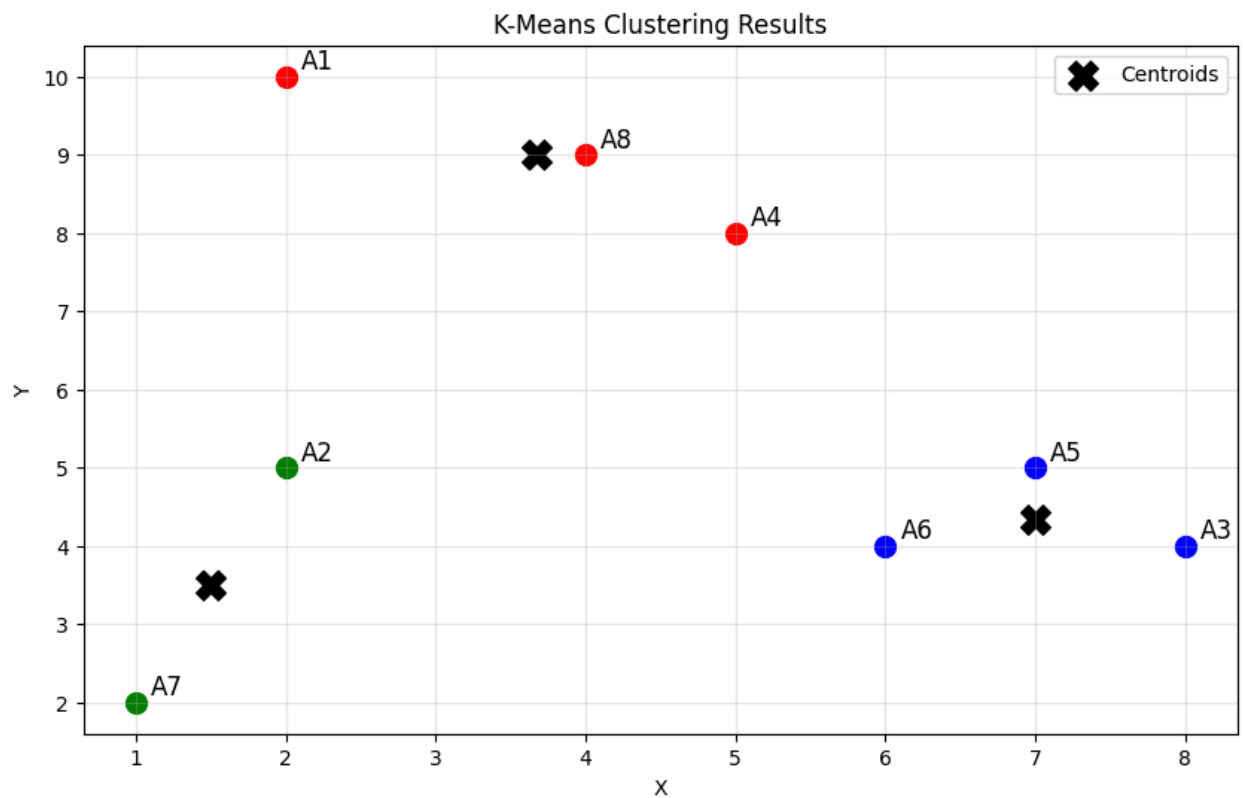
# Plot points
for i in range(len(points)):
    cluster = kmeans.labels_[i]
    plt.scatter(points[i, 0], points[i, 1], c=colors[cluster], s=100)
    plt.text(points[i, 0]+0.1, points[i, 1]+0.1, point_names[i], fontsize=12)

# Plot cluster centers
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            c='black', marker='X', s=200, label='Centroids')

plt.xlabel('X')
plt.ylabel('Y')
plt.title('K-Means Clustering Results')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Print detailed results
print("DETAILED RESULTS:")
for cluster_id in range(3):
    indices = np.where(kmeans.labels_ == cluster_id)[0]
    cluster_points = [point_names[i] for i in indices]
    print(f"\nCluster {cluster_id}: {cluster_points}")
    print(f"Center: ({kmeans.cluster_centers_[cluster_id, 0]:.2f}, {kmeans.clu
    for idx in indices:
        print(f" {point_names[idx]}{tuple(points[idx])}")

```



DETAILED RESULTS:

Cluster 0: ['A1', 'A4', 'A8']
Center: (3.67, 9.00)
A1(np.int64(2), np.int64(10))
A4(np.int64(5), np.int64(8))
A8(np.int64(4), np.int64(9))

Cluster 1: ['A3', 'A5', 'A6']
Center: (7.00, 4.33)
A3(np.int64(8), np.int64(4))
A5(np.int64(7), np.int64(5))
A6(np.int64(6), np.int64(4))

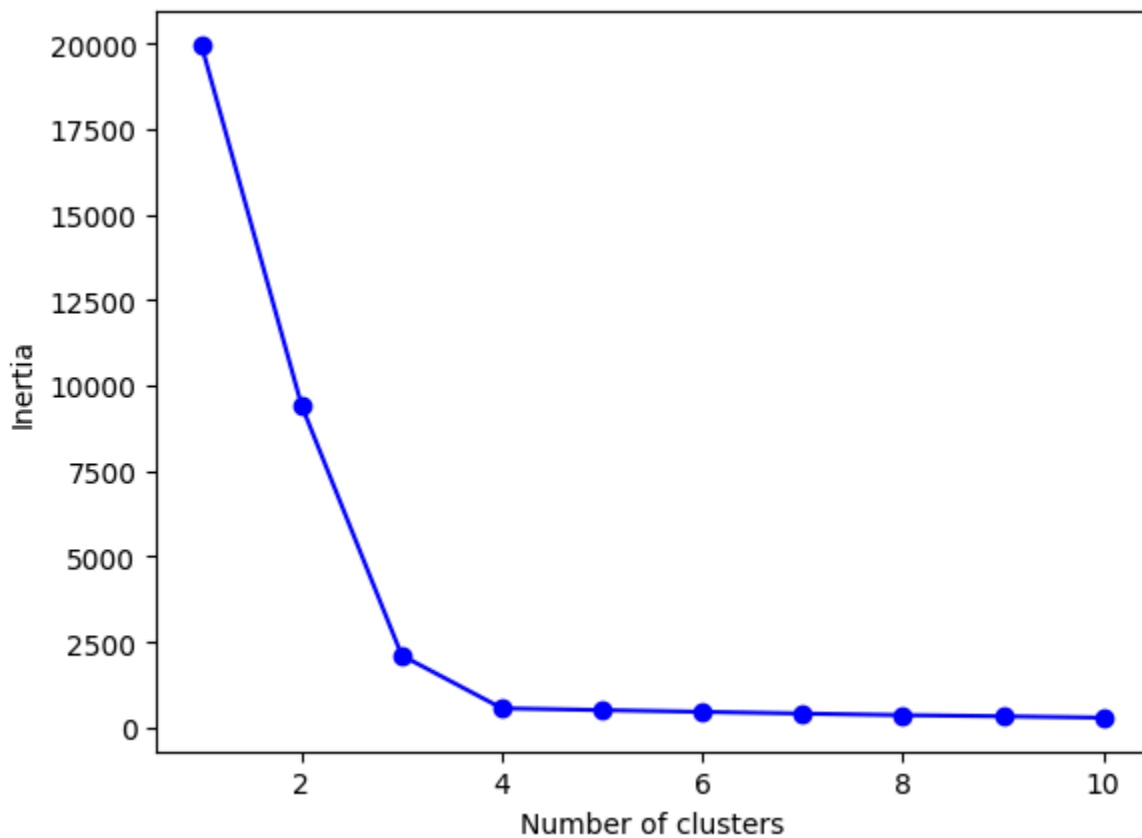
Cluster 2: ['A2', 'A7']
Center: (1.50, 3.50)
A2(np.int64(2), np.int64(5))
A7(np.int64(1), np.int64(2))

```
In [ ]: #elbow method implementation
#import the libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
```

```
In [ ]: #generate synthetic data
X,y = make_blobs(n_samples=300, centers=4, random_state=42)
```

```
In [ ]: #calculate inertia for different values
inertia = []
for k in range(1,11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)
```

```
In [ ]: #plot the curve
plt.plot(range(1,11),inertia,'bo-')
plt.xlabel('Number of clusters')
plt.ylabel('Inertia')
plt.show()
```



```
In [ ]:
```

```
In [ ]:
```

In []:



```
In [ ]: #KMeans++ using make_blobs and without using library
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

def kmeans_plus_plus(X, k):
    n_samples, _ = X.shape
    # Initialize centroids list and randomly select the first centroid
    centroids = [X[np.random.choice(range(n_samples))]]
    for _ in range(1, k):
        # Compute distances from each point to the nearest centroid
        dist_sq = np.array([min([np.inner(c-x, c-x) for c in centroids]) for x in X])

        # Compute the probability of selecting each point as the next centroid
        prob = dist_sq / dist_sq.sum()

        # Select the next centroid based on the computed probabilities
        next_centroid = X[np.random.choice(range(n_samples), p=prob)]
        centroids.append(next_centroid)

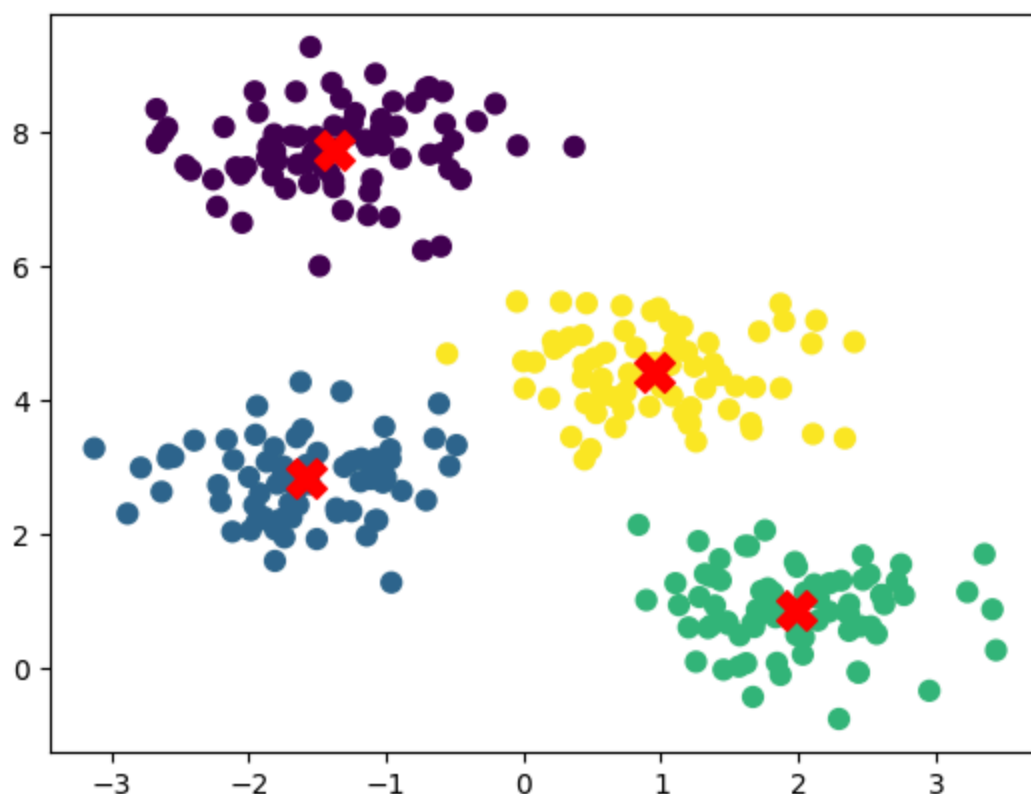
    return np.array(centroids)

def kmeans(X, k, max_iters=300, tol=1e-4):#tol represents tolerance
    # Initialize centroids using KMeans++
    centroids = kmeans_plus_plus(X, k)
    for i in range(max_iters):
        # Assign clusters based on closest centroid
        clusters = np.argmax(np.linalg.norm(X[:, np.newaxis] - centroids,
                                             axis=2), axis=1)

        # Compute new centroids from the clusters
        new_centroids = np.array([X[clusters == j].mean(axis=0) for j in range(k)])
        # Check for convergence
        if np.all(np.linalg.norm(new_centroids - centroids, axis=1) < tol):
            break
        centroids = new_centroids
    return centroids, clusters

# Generate synthetic data using make_blobs
X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# Number of clusters
k = 4
# Run KMeans algorithm
centroids, clusters = kmeans(X, k)
# Plot the results
plt.scatter(X[:, 0], X[:, 1], c=clusters, s=50, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], s=200, c='red', marker='X')
plt.show()
```



```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Generate a dataset using make_blobs
X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# Visualize the dataset
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Dataset with 4 clusters")
plt.show()

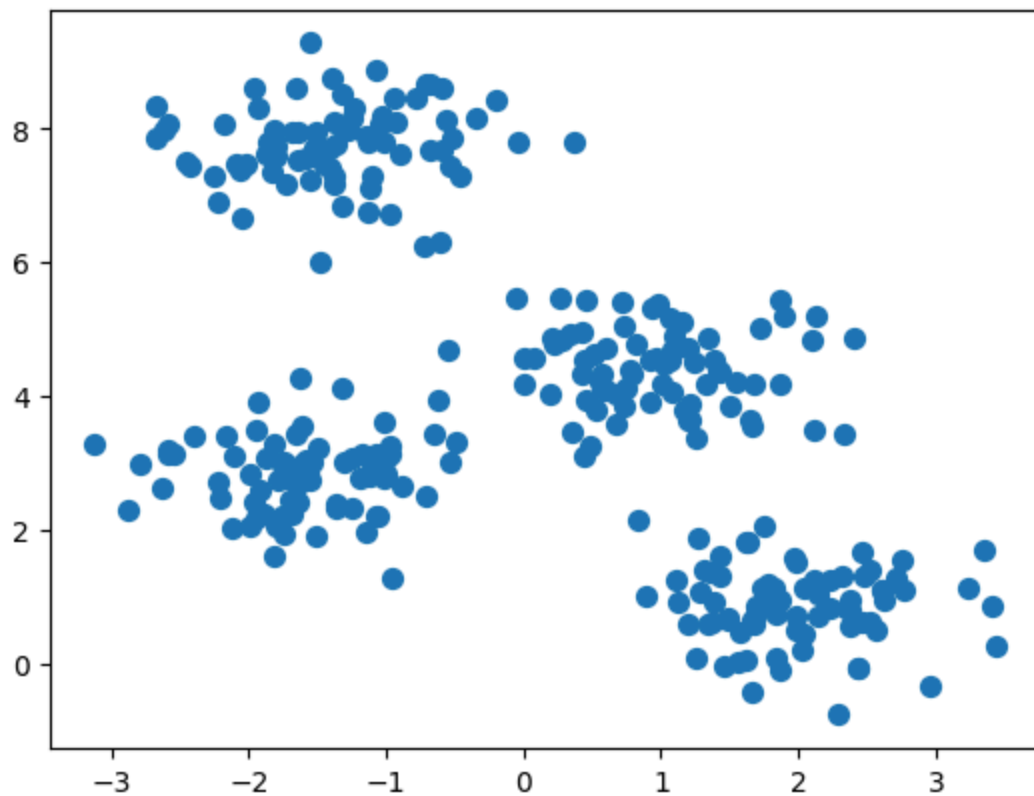
# Initialize and fit KMeans using k-means++ initialization
kmeans = KMeans(n_clusters=4, init='k-means++', n_init=10, max_iter=300, random_state=0)
kmeans.fit(X)

# Predict the cluster labels
y_kmeans = kmeans.predict(X)

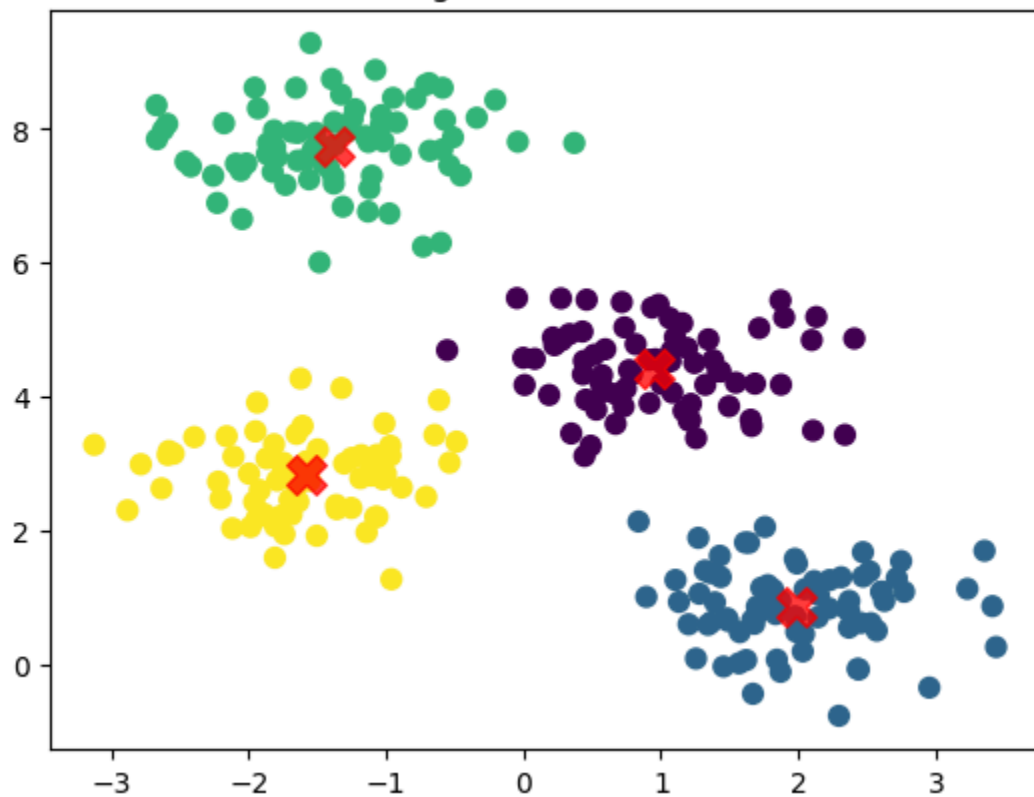
# Plot the clustered data
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')

# Plot the cluster centers
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75, marker='x')
plt.title("K-Means Clustering with k-means++ Initialization")
plt.show()
```

Dataset with 4 clusters



K-Means Clustering with k-means++ Initialization





```
In [5]: # q1 Anomaly Detection using Decision Tree
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import importlib
importlib.reload(np.random)

# Generate synthetic data
np.random.seed(42)
normal_data = np.random.normal(0, 2, (1000, 2))
anomaly_data = np.random.uniform(-20, 20, (50, 2))

# Create labels (0 for normal, 1 for anomalies)
data = np.vstack((normal_data, anomaly_data))
labels = np.array([0] * 1000 + [1] * 50)
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(data, labels, test_size=0.2)

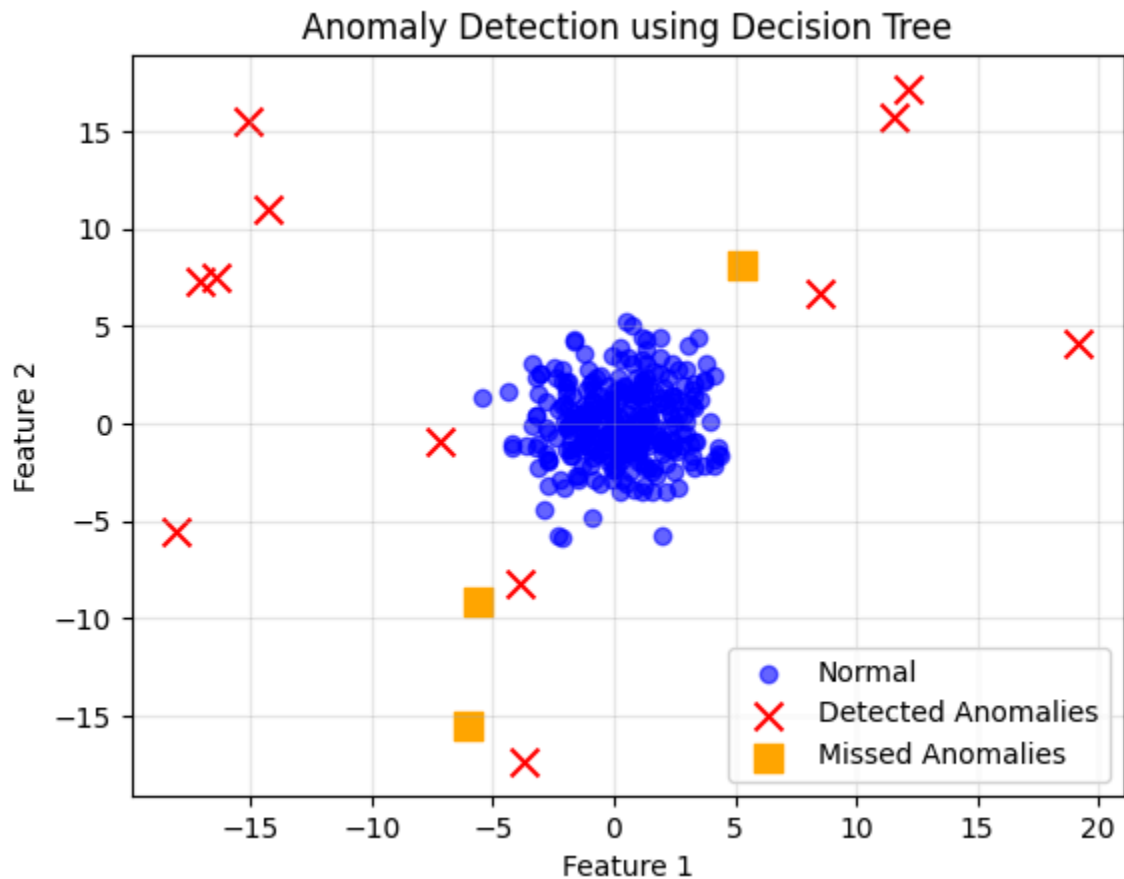
# Fit Decision Tree model
model = DecisionTreeClassifier(max_depth=10, random_state=42)
model.fit(X_train, y_train)

# Predict anomalies
predictions = model.predict(X_test)

# Evaluate model
print("Classification Report:")
print(classification_report(y_test, predictions))
# Plot normal points
normal_mask = (y_test == 0)
plt.scatter(X_test[normal_mask, 0], X_test[normal_mask, 1], c='blue', label='Normal')
# Plot correctly detected anomalies
correct_anomalies = (predictions == 1) & (y_test == 1)
plt.scatter(X_test[correct_anomalies, 0], X_test[correct_anomalies, 1], c='red', label='Correctly detected anomalies')
# Plot missed anomalies (false negatives)
missed_anomalies = (predictions == 0) & (y_test == 1)
plt.scatter(X_test[missed_anomalies, 0], X_test[missed_anomalies, 1], c='orange', label='Missed anomalies')
plt.title("Anomaly Detection using Decision Tree")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	300
1	0.86	0.80	0.83	15
accuracy			0.98	315
macro avg	0.92	0.90	0.91	315
weighted avg	0.98	0.98	0.98	315



```
In [6]: #Q2 Anomaly Detection using K-Means
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Generate synthetic data
np.random.seed=42
normal_data = np.random.randn(1000, 2) * 2 # Normal data points
anomaly_data = np.random.randn(50, 2) * 10 # Anomaly data points

# Combine normal and anomaly data
data = np.vstack((normal_data, anomaly_data))

# Fit K-Means model
num_clusters = 5
model = KMeans(n_clusters=num_clusters)
```

```

model.fit(data)

# Assign data points to clusters
cluster_labels = model.labels_

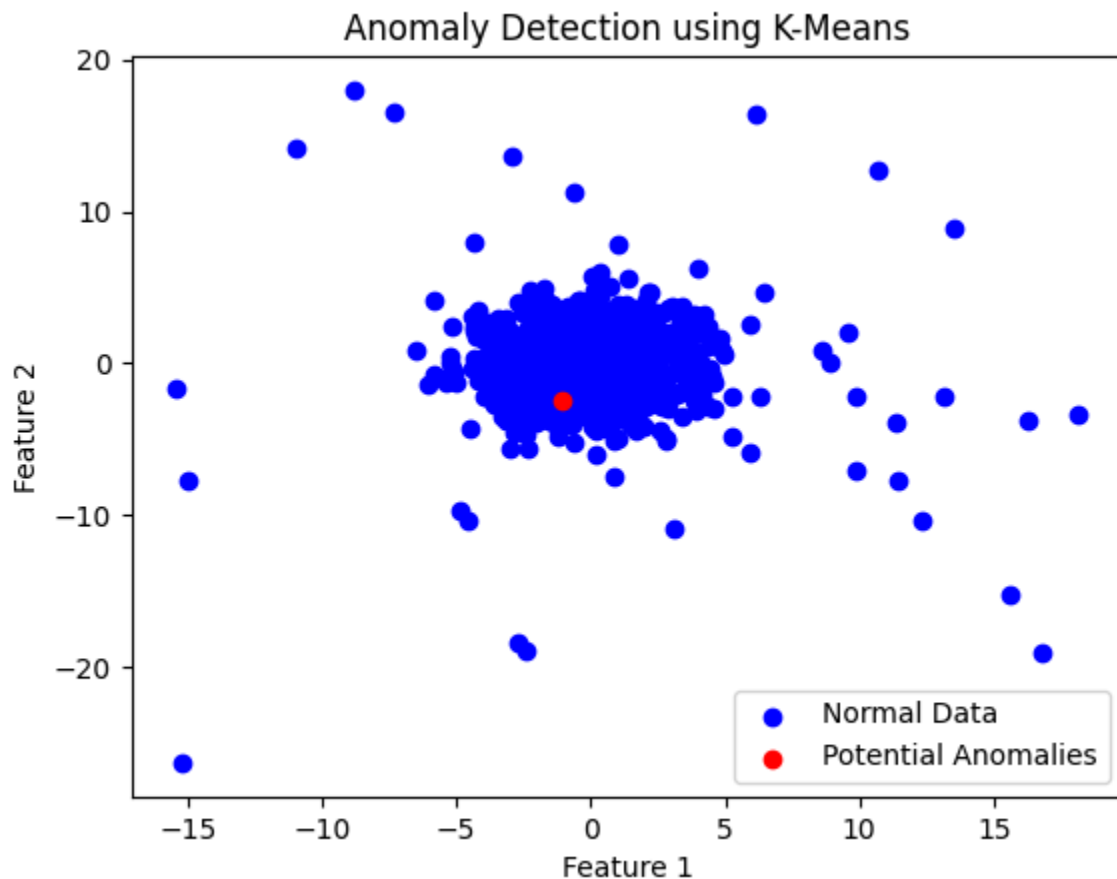
# Calculate cluster sizes
cluster_sizes = np.bincount(cluster_labels)

# Define a threshold for cluster size to identify potential anomalies
anomaly_threshold = np.percentile(cluster_sizes, 5) # 5th percentile cluster

# Identify potential anomalies
potential_anomalies = data[cluster_labels[np.where(cluster_sizes <= anomaly_th

# Plot the data and potential anomalies
plt.scatter(data[:, 0], data[:, 1], c='blue', label='Normal Data')
plt.scatter(potential_anomalies[:, 0], potential_anomalies[:, 1], c='red', lab
plt.legend()
plt.title("Anomaly Detection using K-Means")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()

```



In [8]: #Q3 Apply anomaly detection algorithm using KMeans clustering on make_blobs

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Generate sample data
X, _ = make_blobs(n_samples=300, centers=3, random_state=42)

# Apply KMeans clustering
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X)

# Calculate distances from each point to its cluster center
distances = np.linalg.norm(X - kmeans.cluster_centers_[labels], axis=1)

# Set threshold as 96th percentile (points with distance > 96th percentile are
threshold = np.percentile(distances, 96)
anomalies = distances > threshold

print(f"Total points: {len(X)}")
print(f"Anomalies detected: {np.sum(anomalies)}")
print(f"Threshold distance: {threshold:.4f}")

# Plot results
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1],
            marker='x', s=200, linewidths=3, color='red')
plt.title('KMeans Clustering')

plt.subplot(1, 2, 2)
plt.scatter(X[~anomalies, 0], X[~anomalies, 1], c=labels[~anomalies], cmap='viridis')
plt.scatter(X[anomalies, 0], X[anomalies, 1], color='red', marker='x', s=100,
            plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1],
            marker='x', s=200, linewidths=3, color='red')
plt.title('Anomaly Detection (4th Percentile Threshold)')
plt.legend()

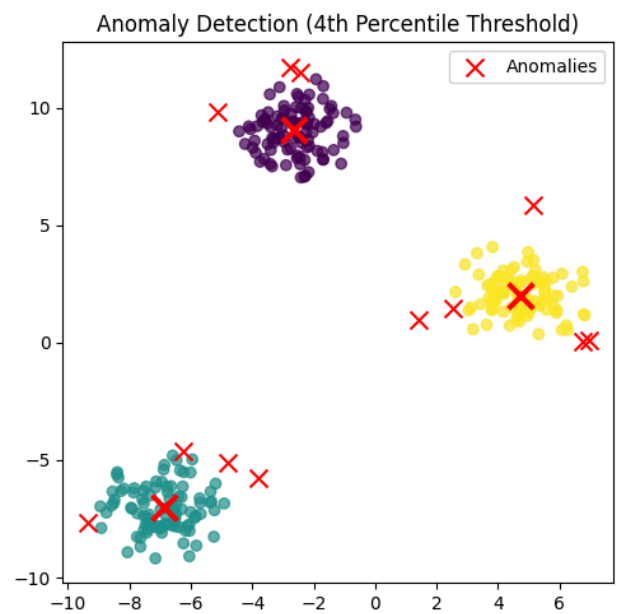
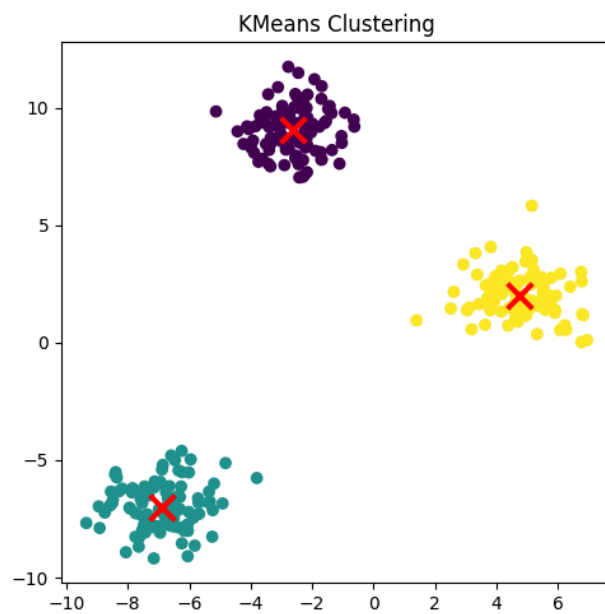
plt.tight_layout()
plt.show()

```

Total points: 300

Anomalies detected: 12

Threshold distance: 2.2930

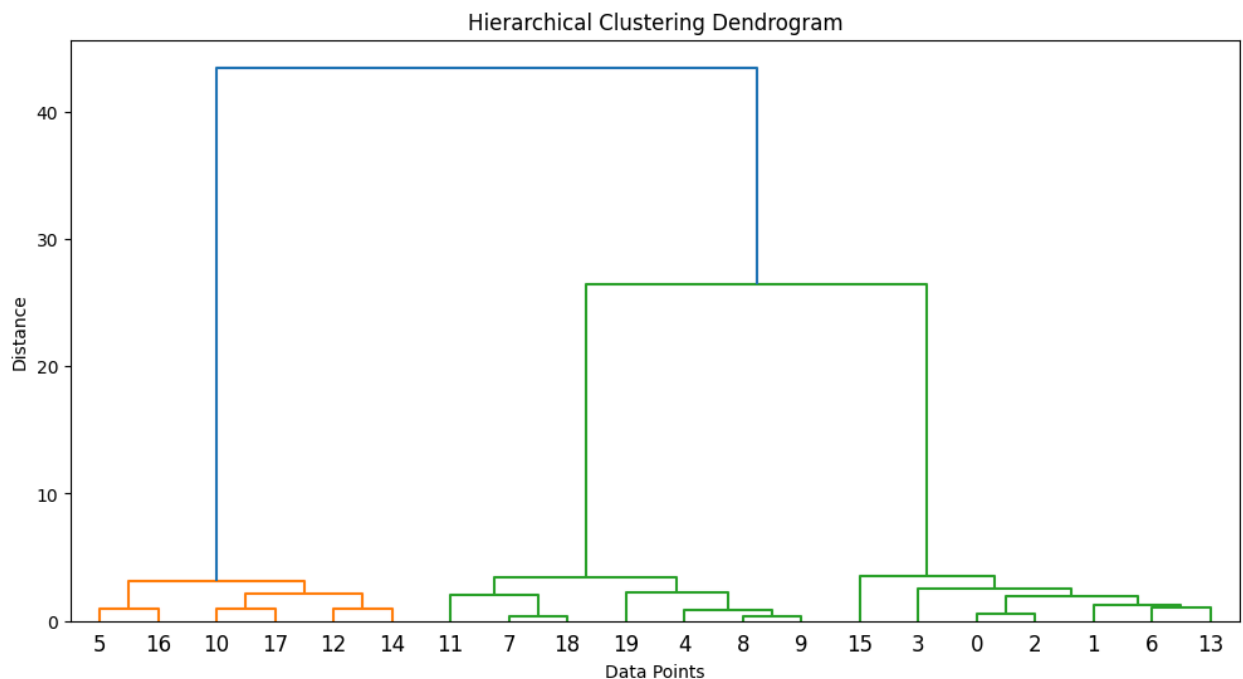


```
In [1]: import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.datasets import make_blobs
import numpy as np

# Generate sample data
X, _ = make_blobs(n_samples=20, centers=3, random_state=42)

# Perform hierarchical clustering
Z = linkage(X, method='ward')

# Plot dendrogram
plt.figure(figsize=(12, 6))
dendrogram(Z)
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Data Points')
plt.ylabel('Distance')
plt.show()
```



```
In [2]: import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from sklearn.datasets import make_blobs

# Generate data
X, _ = make_blobs(n_samples=20, centers=3, random_state=42)

# Perform agglomerative clustering
Z = linkage(X, method='ward')

# Get flat clusters (cut the tree at 3 clusters)
cluster_labels = fcluster(Z, t=3, criterion='maxclust')
```

```

# Create a figure with two subplots
plt.figure(figsize=(15, 5))

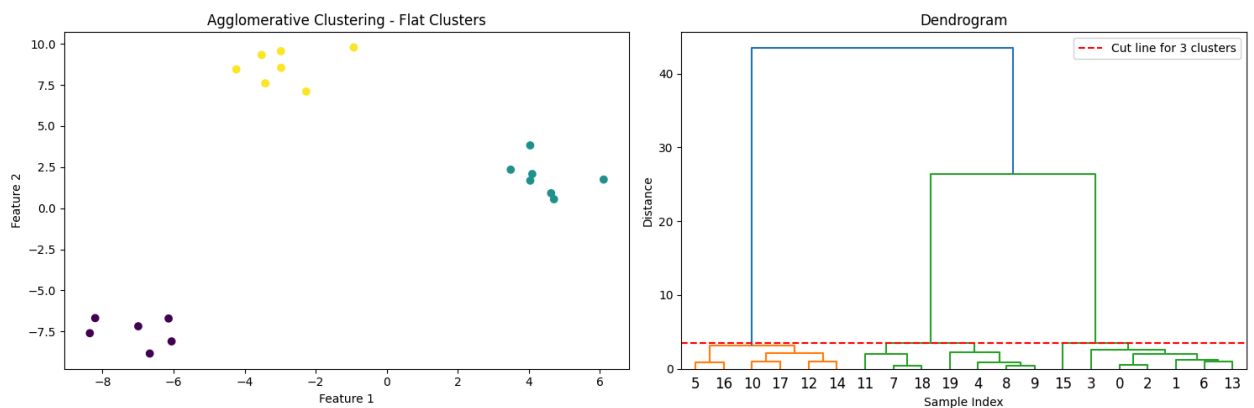
# Plot 1: Flat clusters
plt.subplot(1, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=cluster_labels, cmap='viridis')
plt.title('Agglomerative Clustering - Flat Clusters')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

# Plot 2: Dendrogram
plt.subplot(1, 2, 2)
dendrogram(Z)
plt.title('Dendrogram')
plt.xlabel('Sample Index')
plt.ylabel('Distance')
plt.axhline(y=Z[-3, 2], color='r', linestyle='--', label='Cut line for 3 clusters')
plt.legend()

plt.tight_layout()
plt.show()

print("Cluster assignments:", cluster_labels)

```



Cluster assignments: [3 3 3 3 2 1 3 2 2 2 1 2 1 3 1 3 1 1 2 2]



- Suppose we have the following dataset of 2D points:
- Let's set $\epsilon = 1.5$ and $\text{MinPts} = 3$.
- Points: [(2, 3), (2, 5), (3, 4), (5, 3), (5, 5), (6, 4), (8, 2), (8, 4), (9, 5)]

```
In [ ]: #IMPORT THE LIBRARIES
import numpy as np
from sklearn.cluster import DBSCAN
```

```
In [ ]: #data and parameters
points = np.array([(2,3),(2,5),(3,4),(5,3),(5,5),(6,4),(8,2),(8,4),(9,5)])
eps=1.5
min_samples= 3
```

```
In [ ]: #run dbscan
labels= DBSCAN(eps=eps,min_samples=min_samples).fit_predict(points)
```

```
In [ ]: #show results
print("points:",points.tolist())
print("labels:",labels.tolist())
```

```
points: [[2, 3], [2, 5], [3, 4], [5, 3], [5, 5], [6, 4], [8, 2], [8, 4], [9,
5]]
labels: [0, 0, 0, 1, 1, 1, -1, -1, -1]
```

```
In [ ]: #Let's set  $\epsilon = 1.5$  and  $\text{MinPts} = 3$ .
#Points: [(2, 3), (2, 5), (3, 4), (5, 3), (5, 5), (6, 4), (8, 2), (8, 4), (9,
```

```
In [ ]: #IMPORT the libraries
import numpy as np
from sklearn.cluster import DBSCAN
```

```
In [ ]: #data and parameters
points =np.array([(2, 3), (2, 5), (3, 4), (5, 3), (5, 5), (6, 4), (8, 2), (8,
eps=1.5
min_samples= 3
```

```
In [ ]: #run the dbscan
labels = DBSCAN(eps=eps,min_samples=min_samples).fit_predict(points)
```

```
In [ ]: #show results
print("points:",points.tolist())
print("labels:",labels.tolist())
```

```
points: [[2, 3], [2, 5], [3, 4], [5, 3], [5, 5], [6, 4], [8, 2], [8, 4], [9,
5]]
labels: [0, 0, 0, 1, 1, 1, -1, -1, -1]
```

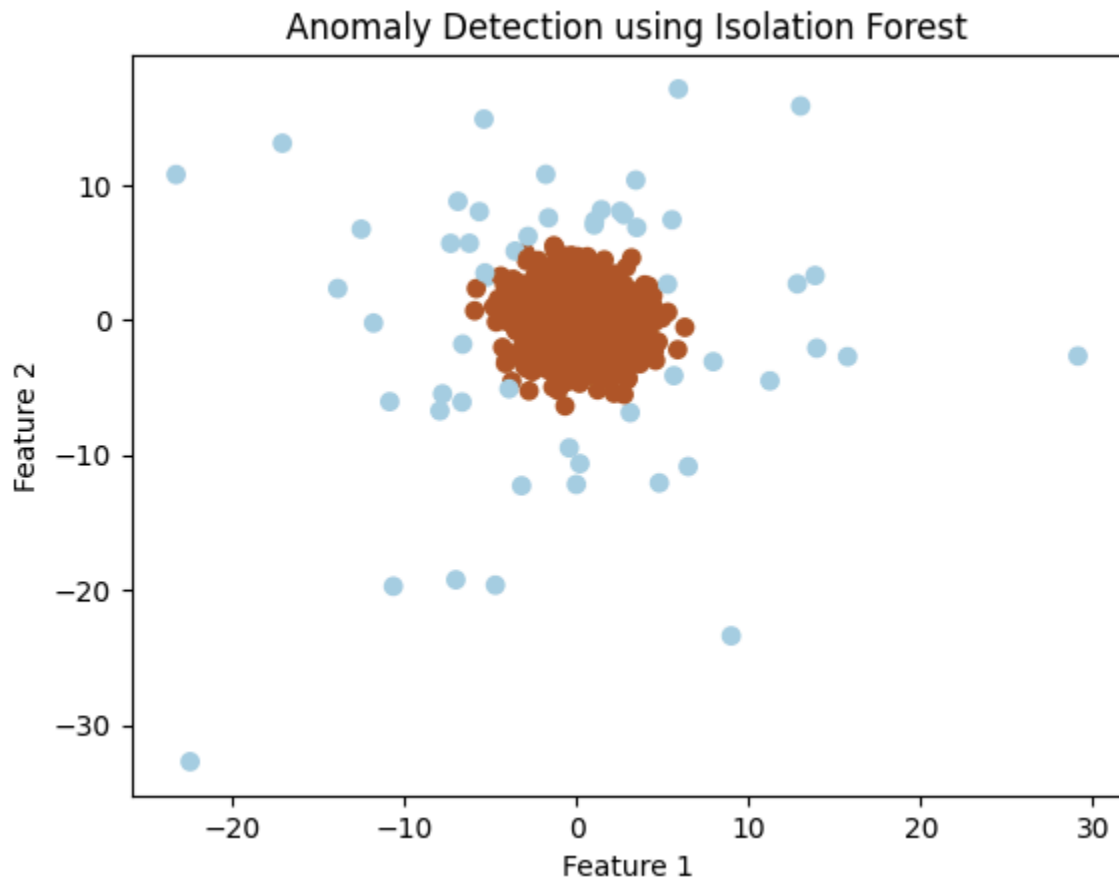
```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest

# Data preparation
normal_data = np.random.randn(1000, 2) * 2
anomaly_data = np.random.randn(50, 2) * 10
data = np.vstack((normal_data, anomaly_data))

# Model training
model = IsolationForest(contamination=0.05)
model.fit(data)

# Prediction and visualization
predictions = model.predict(data)
# Predict anomalies
predictions = model.predict(data)

# Plot the data and detected anomalies
plt.scatter(data[:, 0], data[:, 1], c=predictions, cmap=plt.cm.Paired)
plt.title("Anomaly Detection using Isolation Forest")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```





```
In [1]: #IMPORT the libraries
        from sklearn.decomposition import PCA
        import numpy as np

        # Simple example with 2D data
        data = np.array([[1, 2], [2, 3], [3, 4], [4, 5], [5, 6]])
        print("Original data:\n", data)

        # Apply PCA
        pca = PCA(n_components=1) # Reduce to 1 dimension
        transformed_data = pca.fit_transform(data)

        print("\nTransformed data (1D):\n", transformed_data)
```

Original data:

```
[[1 2]
 [2 3]
 [3 4]
 [4 5]
 [5 6]]
```

Transformed data (1D):

```
[[ -2.82842712]
 [ -1.41421356]
 [  0.         ]
 [  1.41421356]
 [  2.82842712]]
```



```
In [1]: import numpy as np
        from sklearn import metrics
```

```
In [2]: #sample data
        true_labels = [0,0,0,1,1,1,2,2,2]
        pred_labels = [0,0,1,1,1,1,2,2,2]
```

```
In [4]: #calculate ARI
        ari = metrics.adjusted_rand_score(true_labels, pred_labels)
        print(f"ARI:{ari:.3f}")
```

ARI:0.643



```
In [ ]: from sklearn.metrics import normalized_mutual_info_score
import numpy as np
```

```
In [ ]: # Just 2 animals
true_labels = np.array([0, 1]) # Cat, Dog

# Random clustering (wrong)
clustering_wrong = np.array([0, 0]) # Puts both in same cluster
nmi_wrong = normalized_mutual_info_score(true_labels, clustering_wrong)
print(f"Wrong clustering NMI: {nmi_wrong:.1f}") # Output: 0.0

# Perfect clustering
clustering_perfect = np.array([0, 1]) # Separate clusters
nmi_perfect = normalized_mutual_info_score(true_labels, clustering_perfect)
print(f"Perfect clustering NMI: {nmi_perfect:.1f}") # Output: 1.0
```

Wrong clustering NMI: 0.0

Perfect clustering NMI: 1.0



```
In [ ]: # AGENT LEARNS TO MOVE FROM 0 TO POSITION 3
state = 0      #STARTING POINT
goal = 3       #target
reward = 0     #reward counter

while state != goal:
    state += 1  #action:always move right
    print("Moved to:", state)

    if state == goal:
        reward = 1

print("Final Reward:", reward) # reward for reaching the goal
```

```
Moved to: 1
Moved to: 2
Moved to: 3
Final Reward: 1
```

```
In [ ]: #RANDOM ACTION+ REWARD SYSTEM ( TRIAL AND ERROR)

# AGENT does random moves left/right , rewards only if it reaches 4
import random

state = 0
goal = 4

for i in range(6): # perform 6 actions
    action = random.choice([-1, 1]) # -1 for left , +1 for right
    state = max(0, state + action) # stay within 0-4

    print("Step", i, "- State:", state)

    if state == goal:
        print("Reward = +1 (Reached Goal!)")
        break
```

```
Step 0 - State: 0
Step 1 - State: 0
Step 2 - State: 1
Step 3 - State: 0
Step 4 - State: 0
Step 5 - State: 0
```

```
In [ ]: Q = 0
alpha = 0.5
gamma = 0.9

reward = 1
future_Q = 0.8

Q = Q + alpha * (reward + gamma * future_Q - Q)

print("Updated Q-value:", Q)
```

Updated Q-value: 0.8600000000000001

```
In [ ]: state = 0
goal = 3
reward = 3
while state!=goal:
    state+=1
    print("moved to:",state)
    if state==goal:
        reward=1
    print("final reward:",reward)
```

moved to: 1
moved to: 2
moved to: 3
final reward: 1

```
In [ ]: import random
state=0
goal=4
for i in range (6):
    action =random.choice([-1,1])
    state = max(0,state + action)
    print("step",i,"-state:",state)
    if state==goal:
        print("reward=+1(reached goal!)")
        break
```

step 0 -state: 0
step 1 -state: 1
step 2 -state: 2
step 3 -state: 1
step 4 -state: 2
step 5 -state: 1

```
In [1]: Q=0    # initial q value
alpha = 0.5    #learning rate
gamma = 0.9    #discount factor
reward = 1     #suppose reward received
future_q = 0.8 #best future q-value
Q = Q + alpha * (reward + gamma * future_q - Q)
print("updated q-value:",Q)
```

updated q-value: 0.8600000000000001



```
In [1]: # Q-function example for a small grid
import numpy as np

# States: S0, S1, S2
# Actions: 0 = left, 1 = right
Q = np.zeros((3, 2)) # 3 states, 2 actions

# Update Q(s,a) manually
s, a = 1, 1 # state S1, action right
reward = 5
gamma = 0.9
next_state = 2

Q[s, a] = reward + gamma * np.max(Q[next_state])

print("Updated Q-table:\n", Q)
```

Updated Q-table:

```
[[0. 0.]
 [0. 5.]
 [0. 0.]]
```

```
In [2]: # Deterministic policy: select highest Q-value action
import numpy as np

Q = np.array([[1, 3],
              [0, 2],
              [4, 1]])

def deterministic_policy(state):
    return np.argmax(Q[state]) # always same action for same state

for s in range(3):
    print(f"Best action for state {s}:", deterministic_policy(s))
```

```
Best action for state 0: 1
Best action for state 1: 1
Best action for state 2: 0
```

```
In [3]: # Bellman value calculation backward in a chain environment
gamma = 0.9

# V(s3)=1 (goal), other states start at 0
states = ["s1", "s2", "s3"]
V = {"s1": 0, "s2": 0, "s3": 1}

# compute backward: s2 -> s3, s1 -> s2
V["s2"] = gamma * V["s3"]
V["s1"] = gamma * V["s2"]

print(V)
```

```
{'s1': 0.81, 's2': 0.9, 's3': 1}
```



```
In [ ]: #IMPORT the libraries
import numpy as np
#q-table: state * actions
Q = np.zeros((5, 4))    # 5 states  #4 actions
alpha = 0.1
gamma = 0.9
epsilon = 0.1
#simulated rewards for moving to goal from state 4

rewards = np.array([-1, -1, -1, -1, 10])

for episode in range(100):
    state = 0
    while state != 4:
        if np.random.rand() < epsilon:
            action = np.random.randint(0, 4)           #explore
        else:
            action = np.argmax(Q[state])

        next_state = state + 1 if action == 0 else state = 4 # exploit
        reward = rewards[next_state]
    # simplified transition
    Q[state, action] += alpha * (reward + gamma * np.max(Q[next_state]) -
                                state = next_state

print("Learned Q-table:\n", Q)
```

Learned Q-table:

```
[[ 4.52476752  0.05510504 -0.62334545 -0.26848278]
 [ 6.18456281  0.16078108  0.63205558  0.1630096 ]
 [ 7.99713137  0.52419834 -0.1          1.07725484]
 [ 9.99973439  0.79838269  2.07820501  0.64990536]
 [ 0.          0.          0.          0.          ]]
```



```
In [3]: import numpy as np
Q = np.zeros((5, 2))
print(Q)
```

```
[[0. 0.]
 [0. 0.]
 [0. 0.]
 [0. 0.]
 [0. 0.]]
```

```
In [4]: states = 10000
actions = 10
Q = np.zeros((states, actions))
print(Q.nbytes / 1e6, "MB")
```

0.8 MB

```
In [5]: features = 20
state_space = 2 ** features
print("Total states:", state_space)
```

Total states: 1048576

```
In [6]: import numpy as np
actions = np.linspace(-1, 1, 1000)
print("Discretized actions:", len(actions))
```

Discretized actions: 1000

```
In [7]: import random
epsilon = 0.2
if random.random() < epsilon:
    action = random.choice([0, 1])
else:
    action = np.argmax(Q[0])
```

```
In [8]: alpha = 0.1
Q[0, 1] = Q[0, 1] + alpha * (5 - Q[0, 1])
```

```
In [10]: reward = 0
if episode_done:
    reward = 10
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipython-input-1754286828.py in <cell line: 0>()
      1 reward = 0
----> 2 if episode_done:
      3     reward = 10

NameError: name 'episode_done' is not defined
```

```
In [11]: from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
```

```
Dense(24, activation='relu', input_shape=(4,)),  
Dense(2)  
)
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: User Warning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```



```
In [ ]: import pandas as pd

# User data
users = pd.DataFrame({
    'user_id': [1, 2, 3],
    'age': [25, 35, 45],
    'gender': ['M', 'F', 'M']
})

# Item data
items = pd.DataFrame({
    'item_id': [101, 102, 103],
    'category': ['Electronics', 'Books', 'Clothing'],
    'price': [199.99, 9.99, 49.99]
})

# Interaction data
interactions = pd.DataFrame({
    'user_id': [1, 2, 1],
    'item_id': [101, 102, 103],
    'rating': [5, 4, 3]
})
```

```
In [ ]: from sklearn.preprocessing import OneHotEncoder, StandardScaler

# One-hot encode categorical features
encoder = OneHotEncoder()
item_encoded = pd.DataFrame(encoder.fit_transform(items[['category']]).toarray(),
                             columns=encoder.get_feature_names_out(['category']))

# Scale numerical features
scaler = StandardScaler()
items['price_scaled'] = scaler.fit_transform(items[['price']])

# Merge datasets to create user-item features
features = interactions.merge(users, on='user_id').merge(items, on='item_id')
print(features)
```

	user_id	item_id	rating	age	gender	category	price	price_scaled
0	1	101	5	25	M	Electronics	199.99	1.385737
1	2	102	4	35	F	Books	9.99	-0.937410
2	1	103	3	25	M	Clothing	49.99	-0.448327

```
In [ ]: import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Sample movie dataset
data = {
    'title': ['The Matrix', 'Inception', 'Interstellar', 'The Prestige', 'Avatar'],
    'genres': ['Action Sci-Fi', 'Action Sci-Fi', 'Adventure Drama Sci-Fi', 'Drama', 'Action Sci-Fi'],
    'description': [
        'A computer hacker learns about the true nature of his reality.',
        'A thief who steals corporate secrets through dream-sharing technology',
        'A man in space explores the boundaries of human existence.',
        'A man becomes obsessed with uncovering the secrets of a famous playwright.',
        'A group of people fight against a powerful corporation that is destroying the environment.'
    ]
}
```

```

        'A team travels through a wormhole in space to save humanity.',
        'Two magicians engage in a battle to create the ultimate illusion.',
        'A marine on an alien planet becomes torn between two worlds.'
    ]
}

# Create a DataFrame
movies = pd.DataFrame(data)

# Combine genres and description into a single "content" field
movies['content'] = movies['genres'] + ' ' + movies['description']

# Display the dataset
print("Dataset:")
print(movies[['title', 'content']])

# Step 1: Vectorize the movie content
vectorizer = TfidfVectorizer(stop_words='english')
content_matrix = vectorizer.fit_transform(movies['content'])

# Step 2: Compute cosine similarity
cosine_sim = cosine_similarity(content_matrix, content_matrix)

# Step 3: Recommendation function
def recommend_movie(title, cosine_sim=cosine_sim, movies=movies):
    # Get the index of the given movie
    idx = movies[movies['title'] == title].index[0]

    # Get similarity scores for all movies
    sim_scores = list(enumerate(cosine_sim[idx]))

    # Sort movies based on similarity scores
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)

    # Get the top 5 similar movies (excluding the first movie itself)
    top_movies = sim_scores[1:6]

    # Return the titles of the top recommended movies
    recommended_titles = [movies.iloc[i[0]]['title'] for i in top_movies]
    return recommended_titles

# Test the recommendation system
movie_title = "Inception"
recommended_movies = recommend_movie(movie_title)

print(f"\nMovies recommended for '{movie_title}':")
for i, movie in enumerate(recommended_movies, start=1):
    print(f"{i}. {movie}")

```

Dataset:

	title	content
0	The Matrix	Action Sci-Fi A computer hacker learns about t...
1	Inception	Action Sci-Fi A thief who steals corporate sec...
2	Interstellar	Adventure Drama Sci-Fi A team travels through ...
3	The Prestige	Drama Mystery Sci-Fi Two magicians engage in a...
4	Avatar	Action Adventure Fantasy A marine on an alien ...

Movies recommended for 'Inception':

1. The Matrix
2. Interstellar
3. The Prestige
4. Avatar

Explanation of the Code Dataset:

Created a small sample dataset containing title, genres, and description. Content Vectorization:

Combined genres and description into a single text field (content). Used TF-IDF Vectorizer to convert text into numerical vectors. Cosine Similarity:

Calculated pairwise cosine similarity between all movies based on the content vectors. Recommendation Function:

Given a movie title, retrieved its similarity scores with all other movies. Sorted the scores in descending order and returned the top 5 most similar movies.

To make this system more robust:

Use a larger dataset like MovieLens. Add user-based collaborative filtering: Use interaction data (e.g., ratings) to make personalized recommendations. Incorporate embeddings: Use pre-trained models like BERT to process descriptions. Hybrid approach: Combine content-based filtering with collaborative filtering.



```
In [1]: import numpy as np
# 1 = interaction, 0 = no interaction
user_item = np.array([
    [1, 0, 1, 0], # User 1
    [0, 1, 1, 0], # User 2
    [1, 0, 0, 1]  # User 3
])
print(user_item)
```

```
[[1 0 1 0]
 [0 1 1 0]
 [1 0 0 1]]
```

```
In [2]: interaction = 1
if interaction == 1:
    print("User showed interest")
else:
    print("No interaction")
```

User showed interest

```
In [3]: user_interactions = user_item.sum(axis=1)
print(user_interactions)
```

```
[2 2 2]
```

```
In [4]: item_popularity = user_item.sum(axis=0)
print(item_popularity)
```

```
[2 1 2 1]
```

```
In [5]: favorites = item_popularity > 1
print(favorites)
```

```
[ True False  True False]
```

```
In [6]: mean_values = user_item.mean(axis=0)
normalized_data = user_item - mean_values
print(normalized_data)
```

```
[[ 0.33333333 -0.33333333  0.33333333 -0.33333333]
 [-0.66666667  0.66666667  0.33333333 -0.33333333]
 [ 0.33333333 -0.33333333 -0.66666667  0.66666667]]
```

```
In [7]: normalized = (user_item - np.mean(user_item)) / np.std(user_item)
print(normalized)
```

```
[[ 1. -1.  1. -1.]
 [-1.  1.  1. -1.]
 [ 1. -1. -1.  1.]]
```

```
In [9]: clicks = np.array([0, 1, 1, 0])
interested_items = np.where(clicks == 1)
print(interested_items)
```

```
(array([1, 2]),)
```

```
In [10]: boolean_matrix = user_item.astype(bool)
print(boolean_matrix)
```

```
[[ True False  True False]
 [False  True   True False]
 [ True False False  True]]
```

```
In [11]: user_id = 0
recommended_items = np.where(user_item[user_id] == 0)
print("Recommend items:", recommended_items)
```

```
Recommend items: (array([1, 3]),)
```